

Grid

Day 4 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“jumping exactly k squares”*
- ▶ *“minimum number of moves required”*

Programming interpretation:

- ▶ Each cell is a node and each legal jump is an unweighted edge.
- ▶ Because every jump costs one move, BFS gives the shortest number of moves.
- ▶ Unvisited cells keep distance -1, which also handles unreachable targets.

Textbook connection: CP4 4.4 a. On Unweighted Graph: BFS

Core technique: BFS on a jump grid

- ▶ Each cell is a node.
- ▶ The digit in a cell tells the exact jump length.
- ▶ Each legal jump costs one move.

- ▶ All moves have equal cost, so BFS finds the minimum number of jumps.
- ▶ The graph is implicit; neighbors are computed from the current digit.
- ▶ A distance grid records both visited status and shortest distance.

- ▶ Parse the grid as integers.
- ▶ Set $\text{distance}[0][0] = 0$ and push the start cell into a queue.
- ▶ Pop cells in BFS order.
- ▶ For each of four directions, jump exactly d cells.
- ▶ If the target cell is in bounds and unvisited, assign $\text{distance} + 1$ and enqueue it.

- ▶ A cell with value 3 has at most four outgoing moves: up/down/left/right by 3.
- ▶ Visited cells are not enqueued again because BFS already found their shortest path.
- ▶ The answer is distance at the bottom-right cell, or -1 if unreachable.

- ▶ BFS processes states in nondecreasing number of moves.
- ▶ The first time a cell is reached, no shorter path to it can exist.
- ▶ The generated moves are exactly the legal jumps from each cell.

Complexity: $O(RC)$ time and $O(RC)$ memory.

- ▶ Decode each input row before converting characters to integers.
- ▶ Jump length 0 creates no useful new move unless already at the destination.

Source lines: grid.py:42-60

```
import sys
from collections import deque

def main():
    input = sys.stdin.buffer.readline

    rows, cols = map(int, input().split())

    grid = []

    for _ in range(rows):
        row = input().strip().decode()

        grid.append([int(char) for char in row])
```

What this chunk does

- ▶ The grid rows arrive as compact digit strings.
- ▶ Each character is converted into the jump length for that cell.
- ▶ The resulting 2D list supports constant-time access during BFS.

Source lines: grid.py:62-73

```
distance = [[-1] * cols for _ in range(rows)]

queue = deque([(0, 0)])
distance[0][0] = 0

directions = [(-1, 0), (1, 0), (0, -1), (0, 1)] # up # down # left #
right
```

What this chunk does

- ▶ distance stores the best known move count for each cell.
- ▶ The start cell is the top-left corner with distance zero.
- ▶ The directions list covers the four cardinal jumps.

Source lines: grid.py:75-99

```
while queue:
    row, col = queue.popleft()

    jump = grid[row][col]

    for dr, dc in directions:
        new_row = row + dr * jump
        new_col = col + dc * jump

        if new_row < 0 or new_row >= rows or new_col < 0 or new_col >=
cols:
            continue

        if distance[new_row][new_col] != -1:
            continue

        distance[new_row][new_col] = distance[row][col] + 1

        queue.append((new_row, new_col))
```

What this chunk does

- ▶ BFS removes cells in increasing move count.
- ▶ The cell's digit determines the exact jump length.
- ▶ A cell is queued only the first time it is reached, which is its shortest distance.

Source lines: grid.py:101-106

```
print(distance[rows - 1][cols - 1])  
  
main()
```

What this chunk does

- ▶ The bottom-right distance is the required answer.
- ▶ If BFS never reached it, the value remains -1.

Solution file: `practice_contests/day_4/grid.py`

Problem: <https://open.kattis.com/problems/grid>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.